LUMEN CORE · TEAM 2 · 213ECE2463

Self-Healing Factory Sensor Network — Build Guide

Adaptive routing + link-quality estimation across 1 STM32F446RE gateway controller and 4 ESP32 mesh nodes, built in STM32CubeIDE 1.19.0 and Thonny/MicroPython. This page is the running reference for wiring, firmware, and test procedure — keep it open on a second screen while you build.

STM32CubeIDE 1.19.0 | Thonny + MicroPython | ESP-NOW mesh | 1× NUCLEO-F446RE | 4× ESP32 DevKit

01 What you're building

Three ESP32 nodes each watch one factory-style sensor and normally talk straight to a fourth ESP32 (the **Gateway**) over ESP-NOW. Each node scores its own link using a sliding-window **packet delivery ratio**. When that score drops below threshold, the node reroutes its packets through a neighbour node instead of declaring the link dead — that neighbour relay is the "self-healing" part. The Gateway forwards everything over UART to the STM32F446RE, which logs it, forwards a clean CSV stream to a laptop over the same USB cable (via ST-LINK's virtual COM port), and lights its onboard LED whenever any node in the network is currently running on a relayed path.

> WHY THIS SPLIT
>
> The STM32 only ever speaks wired UART to one ESP32 — it never joins the wireless mesh itself. That keeps the CubeIDE firmware to "receive a line, log it, forward it" and puts all the routing intelligence in MicroPython on the ESP32s, which is where ESP-NOW actually lives.

02 Bill of materials

CORE HARDWARE

QTY	PART	USED FOR

QTY	PART	USED FOR
1	STM32 Nucleo-F446RE	Gateway controller / PC bridge
4	ESP32 DevKit (30 or 38-pin, any common variant)	1 mesh Gateway + 3 sensor nodes
1	DHT22 / AM2302 temperature-humidity sensor	Node A
1	SW-420 vibration sensor module	Node B
1	MQ-2 gas / smoke sensor module	Node C
1	IR flame sensor module	Gateway ESP32
4	5 mm LED + 220Ω resistor	Per-node "relaying / self-healing active" indicator
1	10 kΩ resistor	DHT22 data-line pull-up (skip if your module already has one)
5	Small breadboards	One per board
~40	Jumper wires (M-M / M-F)	Sensor + UART wiring
1	USB Mini-B cable	Nucleo ↔ PC (power, flashing, ST-LINK VCP)
4	USB cables matching your ESP32 boards	Independent power for each ESP32
1	4+ port USB hub or power bank	Powers the 4 ESP32 boards without hogging laptop ports

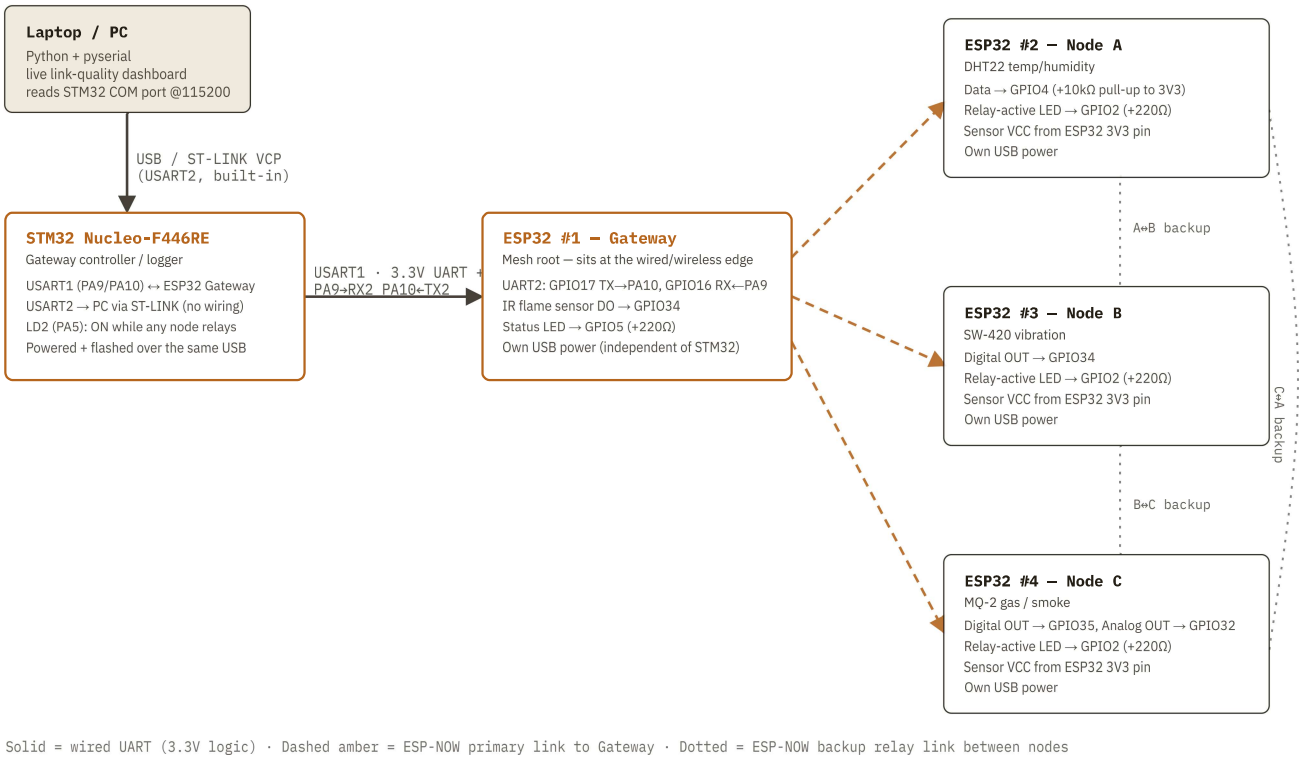
SOFTWARE

TOOL	ROLE
STM32CubeIDE 1.19.0	Firmware for the STM32F446RE (UART gateway + LED logic)
Thonny IDE	Flashing MicroPython and writing/uploading node code to all 4 ESP32s
MicroPython firmware (ESP32 generic build)	Runtime on every ESP32, gives you the `espnow` module
Python 3 + `pyserial`	Laptop-side live dashboard reading the STM32's COM port

> 3.3 V ONLY

> Power every sensor module's VCC from the ESP32's **3.3 V** pin, not 5 V. ESP32 GPIOs are not 5 V-tolerant, and a 5 V-driven digital output from a comparator module (SW-420, MQ-2) will feed 5 V straight into a 3.3 V input pin. Running the modules at 3.3 V keeps their outputs safely in range — you lose a little sensitivity, not function.

03 System & wiring diagram



End-to-end system: wired UART chain on the left (PC → STM32 → Gateway ESP32), wireless ESP-NOW mesh on the right (Gateway + three sensor nodes, with a backup relay ring between nodes for self-healing reroutes).

Per-board pin reference

STM32F446RE (Nucleo)

USART1 TX	PA9 → ESP32 Gateway GPIO16 (RX2)
USART1 RX	PA10 ← ESP32 Gateway GPIO17 (TX2)
GND	Common with Gateway ESP32 GND
LD2	PA5, onboard — self-healing indicator
USART2	PA2/PA3, pre-wired to ST-LINK VCP — do not rewire

ESP32 #1 — Gateway

UART2 TX	GPIO17 → STM32 PA10
UART2 RX	GPIO16 ← STM32 PA9
Flame DO	GPIO34
LED	GPIO5 + 220Ω → GND
Power	Own USB (5V→3V3 onboard regulator)

ESP32 #2 — Node A

DHT22 data	GPIO4 (+10kΩ pull-up to 3V3)
LED	GPIO2 + 220Ω → GND
Sensor VCC	3V3 pin (not 5V)
Power	Own USB

ESP32 #3 — Node B

SW-420 DO	GPIO34
LED	GPIO2 + 220Ω → GND
Sensor VCC	3V3 pin
Power	Own USB

ESP32 #4 — Node C

MQ-2 DO	GPIO35
MQ-2 AO	GPIO32 (ADC1, optional)
LED	GPIO2 + 220Ω → GND
Sensor VCC	3V3 pin

04 Order of operations

Build and test each layer before adding the next — do not wire all five boards up and hope. This order matches the project's own 8-week plan (see §9).

1. **Flash MicroPython onto all 4 ESP32 boards**

   Get the interpreter and `espnow` module in place before writing any node logic.

2. **Bring up STM32↔Gateway UART link**

   Prove the wired half first: STM32 receives whatever the Gateway ESP32 sends and mirrors it to the PC terminal.

3. **Bring up one sensor node talking directly to the Gateway**

   Confirm ESP-NOW pairing and a clean end-to-end path before adding routing logic.

4. **Add link-quality tracking and the backup relay logic**

   Roll it out to all 3 nodes, then deliberately break a link to watch it reroute.

5. **Wire up the sensors and status LEDs for real**

   Swap the placeholder `read_sensor()` stubs for the real DHT22 / SW-420 / MQ-2 / flame readings.

6. **Build the PC dashboard and run the fault-injection test**

Power off a node, move it out of range, or wrap it in foil — watch the LED and dashboard confirm the reroute, then recover.

---

05 Step 1 — Flash MicroPython (Thonny)

1. **Install Thonny and the ESP32 tools**

   Install Thonny (thonny.org). It bundles `esptool`, which is all you need to flash MicroPython — no separate installer required.

2. **Download the MicroPython ESP32 firmware**

   Grab the latest "ESP32 generic" `.bin` from micropython.org/download/ESP32_GENERIC/. Any node board can use the same firmware image.

3. **Erase flash**

   In Thonny: *Run → Select interpreter → MicroPython (ESP32)*, pick the board's COM/serial port, open the "Install or update firmware" dialog, and erase flash first (checkbox in that dialog, or run `esptool.py --port COMx erase_flash` from a terminal).

4. **Write the firmware**

   In the same dialog, select the downloaded `.bin` and install. Repeat for all 4 ESP32 boards — do this one board at a time, each on its own USB port.

5. **Confirm each board boots to a MicroPython REPL**

   Open Thonny's Shell for that port; you should see `>>>`. Run `import espnow` — no error means you're ready.

6. **Read off each board's MAC address**

   Run the snippet below on **each** of the 4 boards and write down the result — you'll hardcode these into every node's peer list.

THONNY SHELL — RUN ON EVERY BOARD

```
import network
sta = network.WLAN(network.STA_IF)
sta.active(True)
print(sta.config('mac'))
```

> KEEP A MAC TABLE
>
> Label your 4 boards physically (tape + marker: G, A, B, C) the moment you read their MAC addresses. Mixing them up later is the single most common source of "ESP-NOW just doesn't arrive" bugs.

---

06 Step 2 — STM32CubeIDE 1.19.0: Gateway controller firmware

1 Create the project

Launch STM32CubeIDE 1.19.0 → *File* → *New* → *STM32 Project*. In the Board Selector tab, search `NUCLEO-F446RE` and select it. Name the project `LumenCore_Gateway` and finish — accept the prompt to initialize all peripherals with their default mode (this auto-configures USART2 as the ST-LINK VCP and PA5/LD2, PC13/user button).

2 Enable USART1 for the ESP32 link

In the generated `.ioc` file's Pinout view, expand *Connectivity* → *USART1* and set Mode to **Asynchronous**. Confirm PA9 auto-assigns as USART1_TX and PA10 as USART1_RX (click each pin manually if it doesn't).

3 Match the baud rate

In the USART1 Configuration panel, set Baud Rate to `115200`, Word Length 8 bits, Parity None, Stop Bits 1 — this must match the Gateway ESP32's UART settings exactly.

4 Turn on the USART1 interrupt

Still in USART1 configuration, open the *NVIC Settings* tab and check **USART1 global interrupt**. This is what lets the MCU receive bytes without polling.

5 Sanity-check the clock tree

Open the *Clock Configuration* tab. The board-selector defaults are fine for this project — just confirm there's no red/error cell; click "Resolve Clock Issues" if CubeIDE flags one.

6 Generate code

Save the `.ioc` (Ctrl+S) or click the yellow gear "Generate Code" icon. CubeIDE scaffolds `main.c`, `usart.c`, and `gpio.c` with HAL init calls already in place.

7 Add the receive buffer and callback

Open `Core/Src/main.c`. Add the variables inside the first `/* USER CODE BEGIN PV */` … `/* USER CODE END PV */` block, and the callback inside `/* USER CODE BEGIN 4 */`

… `/* USER CODE END 4 */` — CubeIDE preserves anything inside these markers on regeneration, so never edit outside them.

CORE/SRC/MAIN.C — USER CODE BEGIN PV

```
uint8_t rxByte;
#define RX_BUF_LEN 64
char lineBuf[RX_BUF_LEN];
uint8_t lineIdx = 0;
```

CORE/SRC/MAIN.C — USER CODE BEGIN 2 (RIGHT AFTER MX_USART1_UART_INIT())

```
HAL_UART_Receive_IT(&huart1, &rxByte, 1);   // arm the first byte
```

CORE/SRC/MAIN.C — USER CODE BEGIN 4

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART1)
  {
    if (rxByte == '\n' || lineIdx >= RX_BUF_LEN - 1)
    {
      lineBuf[lineIdx] = '\0';

      /* Mirror the line straight to the PC over the ST-LINK VCP (USART2) */
      HAL_UART_Transmit(&huart2, (uint8_t*)lineBuf, strlen(lineBuf), 100);
      HAL_UART_Transmit(&huart2, (uint8_t*)"\r\n", 2, 10);

      /* Packet format from the Gateway ESP32: N,,,
         D = direct to gateway, R = relayed via a neighbour (self-healing active)
      if (strchr(lineBuf, 'R') != NULL)
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);    /* LD2 ON  */
      else
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);  /* LD2 OFF */

      lineIdx = 0;
    }
    else
    {
      lineBuf[lineIdx++] = rxByte;
    }
    HAL_UART_Receive_IT(&huart1, &rxByte, 1);   /* re-arm for the next byte */
  }
}
```

1 Build

Project → Build All (or the hammer icon). Fix any red squiggles — the most common one is a missing `#include <string.h>` for `strchr` / `strlen`, which goes in the `USER CODE BEGIN Includes` block.

2 **Flash and run**

Connect the Nucleo's USB (this both powers it and exposes the ST-LINK debugger + VCP). Click the green Run icon (or Debug, then Resume). The onboard ST-LINK flashes the binary automatically.

3 **Open a serial terminal on the VCP port**

In Device Manager (Windows) or `ls /dev/tty.*` / `/dev/ttyACM*` (macOS/Linux), find the ST-LINK Virtual COM Port. Open it in CubeIDE's built-in terminal, PuTTY, or Tera Term at 115200-8-N-1. You won't see anything yet until Step 3's Gateway ESP32 is talking — that's expected.

---

07 Step 3 — ESP32 mesh firmware (Thonny / MicroPython)

Every node runs the same shape of program: join Wi-Fi station mode (needed only to activate the radio — you never connect to an access point), start `espnow`, register its peers by MAC address, and loop. Sensor nodes send readings and track their own delivery ratio; the Gateway just listens and forwards to UART. Paste each file into Thonny, edit the MAC placeholders using the table you built in Step 1, and save it to the matching board as `main.py` so it runs automatically on power-up.

GATEWAY.PY — SAVE AS MAIN.PY ON ESP32 #1

```
import network, espnow, time
from machine import UART, Pin

sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.disconnect()

e = espnow.ESPNow()
e.active(True)

# Peers: the 3 sensor nodes, so their relayed / probe packets are accepted
NODE_A = b'\xAA\xAA\xAA\xAA\xAA\xAA'   # <-- replace with Node A's real MAC
NODE_B = b'\xBB\xBB\xBB\xBB\xBB\xBB'   # <-- replace with Node B's real MAC
NODE_C = b'\xCC\xCC\xCC\xCC\xCC\xCC'   # <-- replace with Node C's real MAC
for mac in (NODE_A, NODE_B, NODE_C):
    e.add_peer(mac)
```

```
uart = UART(2, baudrate=115200, tx=17, rx=16)   # -> STM32 PA10/PA9
led  = Pin(5, Pin.OUT)                          # relay-active indicator

def on_recv(e):
    for host, msg in e:
        if msg is None:
            continue
        line = msg.decode()
        uart.write(line + "\n")
        led.value(1 if line.endswith(",R") else 0)

e.irq(on_recv)

# Optional: also read the local flame sensor and report it as node "G"
flame = Pin(34, Pin.IN)
seq = 0
while True:
    seq += 1
    uart.write("NG,{},{},D\n".format(seq, flame.value()))
    time.sleep(3)
```

NODE_COMMON.PY LOGIC — THE SHAPE ALL 3 SENSOR NODES SHARE

```
import network, espnow, time
from machine import Pin

sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.disconnect()
e = espnow.ESPNow()
e.active(True)

NODE_ID       = "A"                             # "A" / "B" / "C" per board
GATEWAY_MAC   = b'\xGG\xGG\xGG\xGG\xGG\xGG'      # <-- Gateway's real MAC
NEIGHBOR_MAC  = b'\xNN\xNN\xNN\xNN\xNN\xNN'      # <-- backup relay peer's MAC
e.add_peer(GATEWAY_MAC)
e.add_peer(NEIGHBOR_MAC)

led = Pin(2, Pin.OUT)          # relay-active indicator
WINDOW, THRESHOLD = 10, 0.7
window = []
use_relay = False

def on_recv(e):
    # Forward anything a neighbour hands us straight to the gateway
    for host, msg in e:
        if msg and host == NEIGHBOR_MAC:
            e.send(GATEWAY_MAC, msg, True)
e.irq(on_recv)
```

```
def read_sensor():
    return 0   # replaced per-node below

seq = 0
while True:
    seq += 1
    val = read_sensor()
    route = "R" if use_relay else "D"
    packet = "N{},{},{},{}".format(NODE_ID, seq, val, route).encode()
    target = NEIGHBOR_MAC if use_relay else GATEWAY_MAC
    ok = e.send(target, packet, True)

    window.append(ok)
    if len(window) > WINDOW:
        window.pop(0)
    pdr = sum(window) / len(window)

    if not use_relay and pdr < THRESHOLD and len(window) >= 5:
        use_relay, led_state = True, 1
    elif use_relay and e.send(GATEWAY_MAC, b'PROBE', True):
        use_relay, led_state, window = False, 0, [True]
    else:
        led_state = 1 if use_relay else 0
    led.value(led_state)

    time.sleep(2)
```

Drop in the real sensor read for each board, replacing the `read_sensor()` stub above:

PER-NODE SENSOR READ

NODE	SENSOR	READ_SENSOR() BODY
A	DHT22	`dht.measure(); return dht.temperature()` (needs `import dht; dht = dht.DHT22(Pin(4))` at the top)
B	SW-420	`return Pin(34, Pin.IN).value()` — 1 while vibration is detected
C	MQ-2	`return Pin(35, Pin.IN).value()`, or `ADC(Pin(32)).read()` for a raw analog level

> WHY A RING, NOT A FULL MESH
>
> Each node only needs one backup neighbour (A→B→C→A) to demonstrate self-healing — it keeps the peer list short and the MAC bookkeeping manageable. If you want every node reachable through every other node, add the remaining `add_peer` calls and extend `on_recv` to also relay packets it doesn't recognise as its own.

08 Step 4 — PC dashboard

1. **Install pyserial**

   `pip install pyserial` in a terminal on your laptop.

2. **Find the STM32's COM port**

   Same ST-LINK VCP port you used for the serial terminal in Step 2 — close that terminal first so the port is free.

3. **Run the dashboard script**

   Save the script below, edit the port name, and run it while the whole network is powered up.

DASHBOARD.PY

```
import serial

PORT = "COM5"       # Windows: "COM5" · macOS/Linux: "/dev/tty.usbmodemXXXX"
ser = serial.Serial(PORT, 115200, timeout=1)

print(f"{'NODE':<6}{'SEQ':<8}{'VALUE':<10}STATUS")
while True:
    line = ser.readline().decode(errors="ignore").strip()
    if not line:
        continue
    parts = line.split(",")
    if len(parts) != 4:
        continue
    node, seq, val, route = parts
    status = "RELAY (self-healing active)" if route == "R" else "direct"
    print(f"{node:<6}{seq:<8}{val:<10}{status}")
```

09 Step 5 — Fault-injection test

This is the demonstration the whole project is built to show, and it maps directly to the report's Testing phase.

1. **Baseline**

   Power all 5 boards. Confirm the dashboard shows all three nodes as `direct` and LD2 on the Nucleo stays off.

2. **Break one link**

Move Node A far from the Gateway, wrap its antenna area in foil, or briefly unplug it and plug it back in a worse spot. Its packet delivery ratio to the Gateway should drop.

3 **Observe the reroute**

Within a few send cycles, Node A's onboard LED should turn on and the dashboard should show Node A's status flip to `RELAY`, with its data now arriving via Node B. LD2 on the Nucleo turns on at the same time.

4 **Recover**

Restore Node A's direct link. Its periodic probe to the Gateway should succeed, flipping it back to `direct` and turning both LEDs off — this is the "self-healing" recovery, not just failure detection.

5 **Repeat for Node B and Node C**

Confirm each node can both fail over and recover independently.

6 **Log it**

Capture a short screen recording or the dashboard's printed log during one fail/recover cycle — this is your strongest evidence for the report and the viva.

---

10 Timeline (from the project synopsis)

WEEK 1	WEEKS 2–5	WEEKS 6–7	WEEK 8
Design	**Implementation**	**Testing**	**Documentation**

Map this guide onto it: Week 1 = this diagram + BOM ordering. Weeks 2–5 = Steps 1–3 (MicroPython flashing through mesh firmware), one board type at a time. Weeks 6–7 = Step 5's fault-injection cycles, repeated until reroute/recovery is reliable. Week 8 = write up the architecture, PDR/RSSI thresholds you settled on, and the fail/recover log as your results section.

---

11 Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Nucleo terminal shows nothing	Gateway ESP32 UART pins swapped, or baud mismatch	Confirm GPIO17→PA10 and GPIO16←PA9 (crossed, not straight-through), both at 115200
`espnow.send()` always returns `False`	Peer MAC wrong, or peer not added with `add_peer()`	Re-read the MAC with the Step 1 snippet; ESP-NOW silently drops sends to unregistered peers
Node never relays even when moved far away	Threshold/window too forgiving, or sleep interval too long to see a window fill	Lower `THRESHOLD` temporarily to 0.9 to force-trigger and confirm the logic path works, then tune back down
DHT22 read raises an exception	Missing pull-up, or reading faster than the sensor's ~2s cycle	Add the 10kΩ pull-up if your module lacks one; keep the loop's `time.sleep(2)` or longer
MQ-2 / SW-420 output always reads HIGH or always LOW	Module's onboard comparator trimpot not adjusted, or powered at the wrong voltage	Turn the small potentiometer on the module while breathing on it / tapping it until the LED on the module itself responds; confirm 3.3V supply
Two boards keep colliding / overwriting each other's data on the dashboard	Duplicate `NODE_ID` used on two boards	Double check each board's `main.py` has a unique ID matching its physical label

---

Kalasalingam Academy of Research and Education · Dept. of ECE with Elevium · Team LUMEN CORE, Team 2 · AY 2026–2027